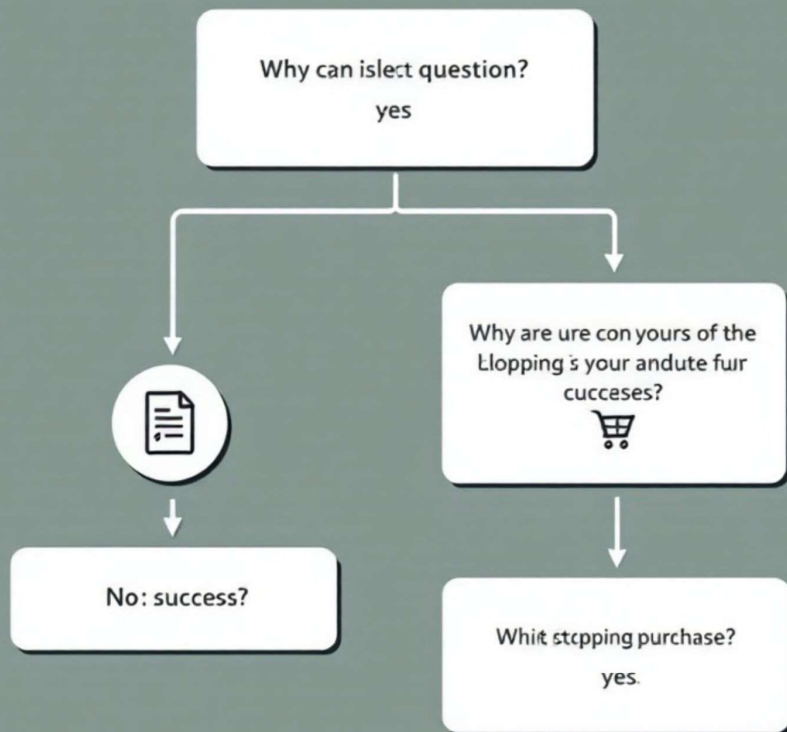




Control del Flujo en Python

En los lenguajes imperativos como Python, el ordenador sigue la secuencia de instrucciones del código y lo ejecuta línea a línea. Denominamos «flujo» de un programa al orden en la ejecución del código. Este orden secuencial puede alterarse mediante diferentes estructuras de control.

La llamada a una función, por ejemplo, lleva la ejecución al código en su interior (el «cuerpo» de la función) para regresar inmediatamente después de dicha llamada y continuar.



Sentencias Condicionales

Los condicionales son estructuras de control que permiten que un programa **tome decisiones basadas en ciertas condiciones**. Estas sentencias nos permiten evaluar expresiones y ejecutar diferentes bloques de código dependiendo de si estas expresiones son verdaderas o falsas. En Python, los condicionales se implementan principalmente a través de las sentencias **if**, **elif** (abreviatura de "else if"), y **else**.



Sentencia if

Ejecuta un bloque de código si una condición es verdadera.

La sintaxis básica es la siguiente:

```
if condicion:
    # bloque de código si la condición es
    verdadera
```



Sentencia else

Ejecuta un bloque de código cuando la condición en el if es falsa.

La sintaxis básica es la siguiente:

```
if condicion:
    # bloque de código si la condición es
    verdadera
else:
    # bloque de código si la condición es
    falsa
```



Sentencia elif

Evalúa múltiples condiciones de manera secuencial. Es una abreviatura de "else if".

La sintaxis básica es la siguiente:

```
if condicion_1:
    # bloque de código si la condicion_1
    es verdadera
elif condicion_2:
    # bloque de código si la condicion_1
    es falsa y la condicion_2 es
    verdadera
else:
    # bloque de código si todas las
    condiciones anteriores son falsas
```

Ejemplos de Sentencias Condicionales

Ejemplo básico if

La sentencia if se utiliza para verificar si un número es mayor que 10:

```
numero = 15
if numero > 10:
    print("El número es mayor que 10")
```

Como la condición es verdadera, veremos en la salida "El número es mayor que 10".

Ejemplo if-else

Podemos agregar un else para manejar el caso en que el número no sea mayor que 10:

```
numero = 5
if numero > 10:
    print("El número es mayor que 10")
else:
    print("El número es menor o igual a 10")
```

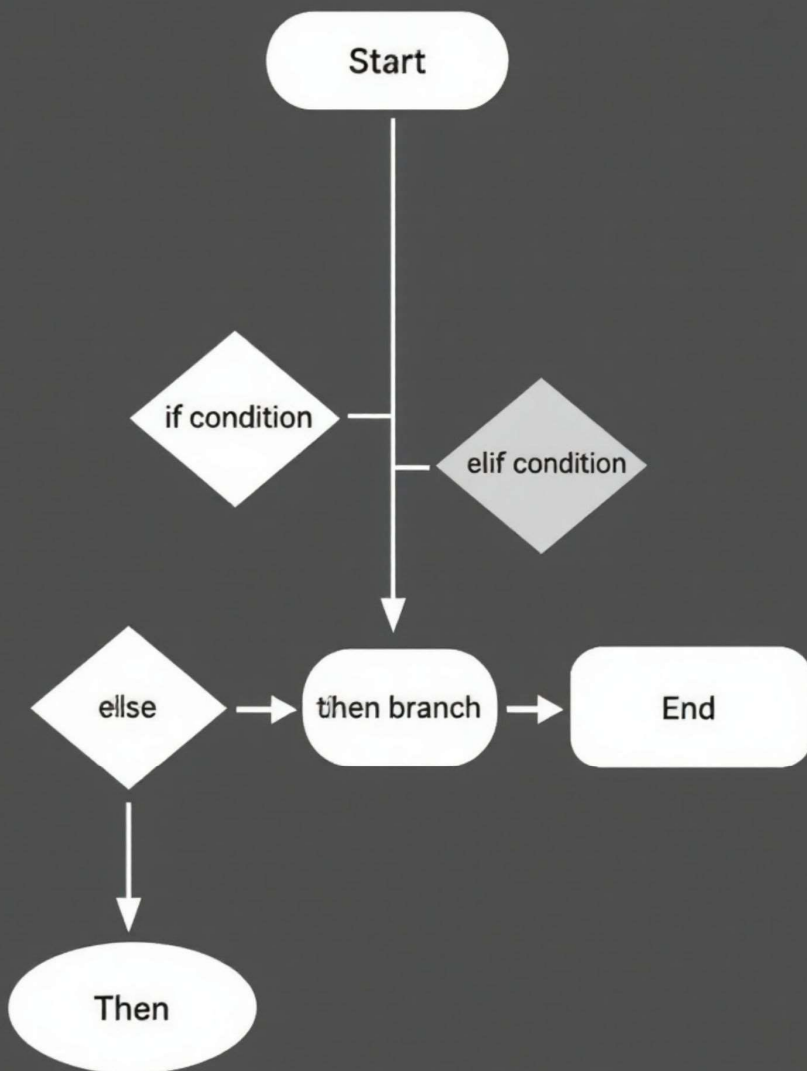
Como numero es 5, se ejecutará el bloque de código dentro del else.

Ejemplo elif

Veamos un ejemplo donde utilizamos elif para clasificar un número en relación con cero:

```
numero = 5
if numero > 0:
    print("El número es positivo")
elif numero < 0:
    print("El número es negativo")
else:
    print("El número es cero")
```

Como numero es 5 y es mayor que 0, se ejecutará el primer bloque de código y veremos en la salida "El número es positivo".



Condiciones Múltiples

Condiciones múltiples

Podemos combinar múltiples condiciones utilizando los operadores and, or y not:

- and: Retorna True si ambas condiciones son verdaderas.
- or: Retorna True si al menos una de las condiciones es verdadera.
- not: Retorna True si la condición es falsa.

Ejemplo condición múltiple

```
x = 10
y = 5
z = 0
if x > y and y > z:
    print("Todas las condiciones son verdaderas")
```

En este caso, las tres condiciones son verdaderas ($x > y$, $y > z$ y $x > z$), por lo tanto, se ejecutará el print.

Match en Python

Python ofrece algo similar al *switch* de otros lenguajes de programación. Se trata de `match`. Cada `case` define un camino posible. El `_` es la opción por defecto, que se ejecuta si la entrada no coincide con ningún caso. `match` permite realizar lo mismo que con múltiples `if/elif`.

Ejemplos

```
hora = 8
match hora:
    case 8:
        print("Desayuno")
    case 14:
        print("Comida")
    case 21:
        print("Cena")
    case _:
        print("No toca comer")
# Desayuno
```

```
hora = 8
if hora == 8:
    print("Desayuno")
elif hora == 14:
    print("Comida")
elif hora == 21:
    print("Cena")
else:
    print("No toca comer")
```

También podemos tener en nuestros `case` múltiples condiciones, donde `|` es interpretado como un `or`.

```
mes = 4
match mes:
    case 12 | 1 | 2: print("Invierno")
    case 3 | 4 | 5: print("Primavera")
    case 6 | 7 | 8: print("Verano")
    case 9 | 10 | 11: print("Otoño")
    case _: print("Error")
# Primavera
```

Es posible hacer *matching* de prácticamente cualquier cosa, como una tupla. Este ejercicio muestra cómo usar `match-case` para realizar **desempaquetado de patrones** con tuplas. El sistema no solo compara valores, sino que extrae datos y los asigna a variables en tiempo de ejecución.

```
coordenada = (0, 1)
match coordenada:
    case (0, 0):
        print("Coordenada en origen")
    case (x, 0):
        print(f"Coordenada en eje x: {x}")
    case (0, y):
        print(f"Coordenada en eje y: {y}")
    case (x, y):
        print(f"Coordenada en: {x}, {y}")
    case _:
        print("Error")
```

```
# Coordenada en eje y: 1
```

```
"""
```

El patrón exige que el primer elemento sea exactamente 0 (verdadero).

El segundo elemento se asigna automáticamente a la variable y.

Por lo tanto, y toma el valor 1 y se ejecuta este bloque.

```
"""
```

Ejemplo para saber si un número es par:

```
try:
    num = int(input("Introduce un número: "))

    match num:
        case 0:
            print("El número es un cero")
        case num if num % 2 == 0:
            print("El número es par")
        case _:
            # Simplificado: si no es cero ni par, obligatoriamente es impar
            print("El número es impar")

except ValueError:
    # Este bloque se activa solo si la conversión a int() falla
    print("Error: No has introducido un número entero válido.")
```

Bucle For en Python

El bucle for se utiliza para **iterar sobre una secuencia** (como una lista, tupla, diccionario, etc.) y ejecutar un bloque de código para cada elemento en esa secuencia. A diferencia de otros lenguajes, con el for de Python no se definen contadores o expresiones que cambien a cada iteración, sino una secuencia sobre la cual la variable contador tomará sus valores.

La sintaxis básica es la siguiente:

```
for elemento in secuencia:  
    # bloque de código a ejecutar para cada elemento
```

Iterar sobre una colección

Recorrer elementos de una lista, tupla o conjunto.

```
frutas = ["manzana", "banana", "cereza"]  
for fruta in frutas:  
    print(fruta)
```

En este caso, el bucle for recorre la lista frutas y en cada iteración, asigna el valor del elemento actual a la variable fruta, la cual es luego impresa. La salida será:

manzana

banana

cereza

Iterar un número de veces

También podemos utilizar el bucle for para realizar una serie de iteraciones específicas, por ejemplo, un número fijo de veces.

Esto se logra utilizando la función `range()`, que genera una secuencia de números en un rango dado. Al combinar `range()` con un bucle for, podemos iterar sobre esta secuencia y ejecutar un bloque de código una cantidad predeterminada de veces.

```
# Ejemplo de bucle for con range(5)
for i in range(5):
    print(i) # Imprime 0, 1, 2, 3, 4
```

En este ejemplo, `range(5)` genera una secuencia de números desde 0 hasta 4 (excluyendo el 5), y el bucle for itera sobre cada uno de estos números, asignándolos a la variable `i` en cada iteración. Luego, se imprime el valor de `i`, resultando en la salida:

0

1

2

3

4

Iterar sobre una cadena de texto

Además de iterar sobre listas y otras secuencias, el bucle for en Python puede utilizarse para iterar sobre otros tipos de datos, como cadenas de texto. Esto es así porque una cadena de texto es una colección de caracteres.

Cuando utilizamos un bucle for con una cadena de texto, cada iteración del bucle recorrerá los caracteres individuales de la cadena, permitiéndonos procesar o manipular cada uno de ellos según sea necesario.

```
# Ejemplo de bucle for con una cadena de texto
for letra in "Python":
    print(letra)
```

En este caso, el bucle for itera sobre cada carácter de la cadena de texto "Python". En cada iteración, el valor del carácter actual se asigna a la variable letra, que luego se imprime. La salida será:

P
y
t
h
o
n

Bucle While en Python



Definición

El bucle while se utiliza para **repetir un bloque de código mientras una condición sea verdadera**.



Sintaxis

```
while condicion:  
    # código a ejecutar mientras la condición sea verdadera
```



Precaución

Es importante tener cuidado con las condiciones en los bucles while. Si la condición nunca se vuelve falsa, el **bucle se ejecutará indefinidamente**.

Un ejemplo común es contar de 1 a 5:

```
contador = 1  
while contador <= 5:  
    print(contador)  
    contador += 1
```

En este caso, el bucle while se **ejecutará mientras la condición sea verdadera**, es decir, mientras contador sea menor o igual a 5. En cada iteración, se imprime el valor de contador y luego se incrementa en 1. La salida será:

```
1  
2  
3  
4  
5
```

Ejemplo Práctico: Cálculo del Factorial

Definición

El factorial de un número natural resulta de multiplicar ese número por todos los números inferiores a él hasta el 1.

Bucle

El bucle termina cuando el número alcanza el valor 1.

Implementación

Usamos un acumulador (variable acc) para construir el resultado final mientras decrementamos el número.

Resultado

El factorial de 4 (se escribe 4!) es $4 * 3 * 2 * 1 = 24$.

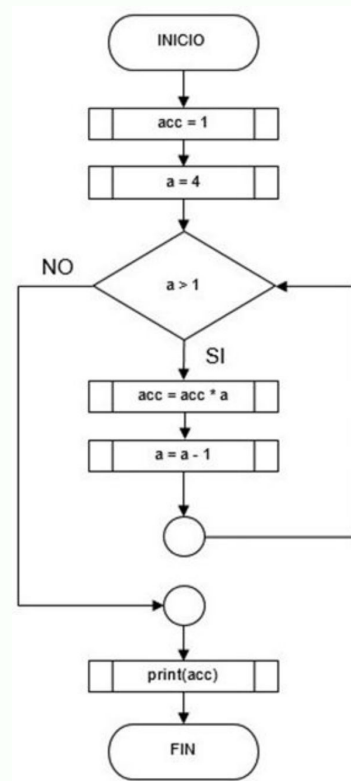
```
a = 4
acc = 1
while a > 1:
    acc = acc * a
    a = a - 1
print(acc) # Resultado: 24
```

Esta sería la «traza» de valores en cada iteración del bucle:

- Inicialización: a: 4, acc: 1
- Fin iteración 1: a: 3, acc: 4
- Fin iteración 2: a: 2, acc: 12
- Fin iteración 3 a: 1, acc: 24
- Fin de bucle (pues la condición $a > 1$ ya no se cumple)

La variable `acc` nos sirve de «acumulador». Un acumulador es una variable sobre la cual se construyen el resultado final. La variable `a` contiene el número cuyo factorial queremos conocer. En el cuerpo del «bucle» (así se denominan también las repeticiones) primero actualiza el acumulador multiplicándolo por el valor actual de `a`, y luego se decrementa `a` en una unidad. El bucle termina cuando `a` alcanza el valor 1. En ese momento la ejecución sale del `while` y sigue con el programa. La instrucción final muestra el resultado en pantalla.

El ordinograma de este ejemplo sería:



continue: (drem"dents">; lav)

constinue: (drem"dents">; lav)

contiinue (rext woolds""(law)

continue = continue)

Control con break, continue y pass



break

Permite "romper" un bucle en cualquier momento, terminar las iteraciones y continuar después del mismo. Generalmente **se usa break para abandonar un bucle while o for** cuando se cumple cierta condición. Aquí tenemos un ejemplo:

```
frutas = ["manzana", "banana", "cereza", "sandía", "uva"]

for fruta in frutas:
    print(fruta)
    if fruta == "sandía":
        break
```

En este caso, el bucle for imprimirá cada fruta de la lista hasta que llegue a "sandía", momento en el cual se ejecutará break y el bucle se interrumpirá.

continue

Permite **saltar a la siguiente iteración** sin ejecutar el resto del bloque de código para esa iteración. Útil para omitir ciertos elementos en un bucle. Veamos un ejemplo:

```
numeros = [1, 2, 3, 4, 5]

for numero in numeros:
    if numero % 2 == 0:
        continue
    print(numero)
```

En este ejemplo, el bucle for itera sobre los números en la lista `numeros`. Cuando encuentra un número par (divisible por 2), se ejecuta `continue`, lo que significa que el `print(numero)` no se ejecuta para ese número y se salta a la siguiente iteración.



pass

No hace nada. Útil cuando necesitamos un bloque de código sintácticamente pero aún no hemos implementado su funcionalidad.

```
if contador_clientes == 1000:  
    pass # POR HACER: celebrar nuestro cliente número 1000
```

En el ejemplo anterior reflejamos la consideración de llegar al cliente número 1000 y, aunque no hemos implementado ninguna acción concreta, lo hemos dejado indicado mediante un comentario.

Aunque estas sentencias pueden mejorar la legibilidad del código en determinadas ocasiones, no conviene abusar de ellas porque dificultan seguir el flujo de código.

Sentencia else en bucles while y for

También existe la posibilidad de utilizar else inmediatamente después de un bucle while o for. Esta opción permite ejecutar un bloque de instrucciones determinado en caso de completar todas las iteraciones, es decir, cuando no se produce un break.

```
numeros = [1, 2, 3, 4, 5]

for numero in numeros:
    if numero == 0:
        break
else:
    print("El bucle ha terminado sin encontrar el 0")
```

En este caso, como no hay ningún 0 en la lista numeros, el bucle se ejecutará completamente y al final se imprimirá “El bucle ha terminado sin encontrar el número 0”.

Uso de range() en Python

La función range() en Python genera una **secuencia inmutable de números enteros** en un rango específico. Podemos invocarla de cuatro formas diferentes:

Un argumento

Genera enteros que van desde 0 hasta el anterior al indicado como parámetro. Por ejemplo, range(5) genera [0, 1, 2, 3, 4]. En general, range(n) genera la lista [0, 1, ..., n-1].

```
# Generar una secuencia de 0 a 4 (exclusivo)
for i in range(5):
    print(i)
# Salida: 0, 1, 2, 3, 4
```

Dos argumentos (especificando inicio y fin)

El primer parámetro es el valor inicial y el segundo el valor por debajo del cual deben estar todos los elementos de la lista generados (igual que en el caso anterior). Por ejemplo, range(5, 10) genera la lista [5, 6, 7, 8, 9]. En general, range(m, n) genera la lista [m, m+1, ..., n-1].

```
# Generar una secuencia de 2 a 8 (exclusivo)
for i in range(2, 9):
    print(i)
# Salida: 2, 3, 4, 5, 6, 7, 8
```

Tres argumentos (especificando paso)

Con tres argumentos. Igual que el anterior, pero el tercer parámetro indica el incremento que se produce de un elemento al siguiente: `range(5, 10, 2)` genera la lista `[5, 7, 9]`. En general, `range(m, n, s)` genera la lista `[m, m+s, ..., n-1]`.

```
# Generar una secuencia de 1 a 10 con paso de 2
for i in range(1, 11, 2):
    print(i)
# Salida: 1, 3, 5, 7, 9
```

Orden inverso

Con la función `range()` también podemos generar secuencias inversas, para ello debemos intercambiar los límites y usar un valor negativo como tercer parámetro. Por ejemplo, `range(10, 4, -1)` generaría la lista `[10, 9, 8, 7, 6, 5]`.

```
# Iterar en orden inverso
for i in range(10, 0, -1):
    print(i)
# Salida: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

La función `range()` devuelve un objeto de tipo `range`, que es eficiente en términos de memoria y adecuado para generar secuencias de números grandes.

`range()` es útil en contextos donde se necesita generar **secuencias numéricas “fijas”** y que no varían (por ejemplo en bucles `for`, operaciones matemáticas y manipulaciones de datos).

Si indicamos límites que no se cumplen desde el primer elemento, el resultado es una lista vacía: `range(5, 3)` daría lugar a una lista sin elementos `[]`.

Ejemplos Prácticos con range()

```
01 -  
11 - even_numbers {  
13 - evennumbers = list( range(2,11,) 2)  
13 - print(evennumbers ;  
  
2 - print {  
4 - 4,6,8,10 2  
16 - 4  
10 - } 6  
13 - 6  
20 - 8  
5
```

Crear lista desde range()

En este ejemplo, se muestra cómo utilizar range para crear una lista de números pares del 0 al 9, utilizando la función list para convertir la secuencia generada por range en una lista:

```
numeros_pares = list(range(0, 10, 2))  
print(numeros_pares)  
# Salida: [0, 2, 4, 6, 8]
```

Iterar en paralelo (como lo hace la función zip)

Este ejemplo muestra cómo usar range como si fuera la función zip para iterar en paralelo sobre dos listas, permitiendo acceder a los elementos correspondientes de ambas listas en cada iteración:

```
# ejemplo con zip() para iterar en paralelo sobre dos listas
nombres = ["Ana", "Luis", "María"]
edades = [30, 25, 35]

# Crea un iterador de tuplas
nombre_edad = zip(nombres, edades)

# Imprime las tuplas (o puedes usar un bucle for)
# print(list(nombre_edad)) # ['Ana', 'Luis', 'Maria'] , [30, 25, 35]
for nombre, edad in nombre_edad:
    print(f"{nombre} tiene {edad} años.")

# Salida:
# Ana tiene 30 años.
# Luis tiene 25 años.
# María tiene 35 años.
```

```
# mismo ejemplo usando range() como si fuera zip() para iterar en paralelo sobre dos listas
nombres = ["Ana", "Luis", "María"]
edades = [30, 25, 35]

for i in range(len(nombres)):
    print(f"{nombres[i]} tiene {edades[i]} años.")
# Salida:
# Ana tiene 30 años.
# Luis tiene 25 años.
# María tiene 35 años.
```

Generar índices en un bucle

En este ejemplo se utiliza `range` para generar índices en un bucle, lo cual es útil cuando se necesita acceder a los elementos de una secuencia junto con sus índices:

```
# Usar range() para generar índices en un bucle
palabra = "Python"

for i in range(len(palabra)):
    print(f"Índice {i}: {palabra[i]}")
# Salida: Índice 0: P, Índice 1: y, Índice 2: t, Índice 3: h, Índice 4: o, Índice 5: n
```